

Linguaggi LISP (autore: Vittorio Albertoni)

Introduzione

Il linguaggio di programmazione LISP nacque verso la fine degli anni cinquanta del secolo scorso, quando i computer si programmavano per lo più in linguaggio assembly o, per le applicazioni scientifiche, in linguaggio Fortran.

Erano anche gli anni in cui si stava ormai superando la fase iniziale dello sviluppo dell'intelligenza artificiale, campo nel quale con il linguaggio assembly non si sarebbe andati da nessuna parte e con il linguaggio Fortran, orientato esclusivamente al calcolo numerico, non si riusciva ad elaborare, oltre ai numeri, anche i simboli, come richiesto dall'intelligenza artificiale.

Il linguaggio LISP è nato per rimediare a questo stato di cose.

Verso la metà degli anni cinquanta del secolo scorso Allen Newell, J. C. Shaw e Herbert Simon svilupparono presso la RAND Corporation una tecnica, da loro denominata «list processing», creando le basi per un linguaggio adatto alla manipolazione di liste e simboli, due importanti tipi di dato nel campo dell'intelligenza artificiale, denominato IPL (Information Processing Language).

E' da queste basi che John McCarthy creò il linguaggio LISP, chiamato così per LISt Processing, in omaggio al nome dato alla tecnica dai tre pionieri, che vide la luce alla fine del 1958 e divenne diffusamente utilizzato con la versione 1.5 del 1962.

Già nei primi anni di vita il LISP si dimostrò molto versatile per arrangiamenti di vario tipo: a partire dagli anni sessanta del secolo scorso, al LISP 1.5 di McCarthy si affiancarono lo Standard LISP, il MacLISP sviluppato presso il MIT, il LISP 1100 sviluppato presso la UNIVAC, l'InterLISP sviluppato presso la Xerox e lo StanfordLISP sviluppato presso l'Università di Stanford.

Nel 1970 vede la luce una versione del LISP con l'obiettivo di unificare un po' tutto: si chiama Scheme.

Ma prosegue comunque lo sviluppo di dialetti vari, tanto che attorno al 1980 se ne contano una dozzina.

Ulteriore e forse definitivo tentativo unificante avviene nel 1984 con la nascita del Common LISP, tuttora prevalentemente usato, al quale è dedicata la prima parte di questo manuale.

Degno di nota, ad opera di Lutz Müller nel 1991, lo sviluppo di un fork semplificante del LISP originario, il NewLISP, sintesi di tutto il LISP fino allora noto con selezione di ciò che serve in maniera più ricorrente per fare cose non troppo complicate, comunque potente ma con modesto utilizzo di risorse.

Alla selezione si aggiunge l'arricchimento con funzioni preconfezionate, non presenti nel linguaggio originario, soprattutto nel campo del calcolo numerico (statistico, finanziario, matriciale). Secondo alcuni il NewLISP non si può nemmeno considerare una derivata del LISP ma un nuovo linguaggio che eredita lo stile del LISP.

A NewLISP è dedicata la seconda parte di questo manuale.

Indice

I	Common LISP	4
1	Predisposizione del computer	4
1.1	CMUCL	4
1.2	SBCL	4
1.3	GCL	5
1.4	CLISP	5
2	Come funziona	6
3	Premessa al linguaggio	8
4	Il linguaggio	9
4.1	Tipi di dato	9
4.1.1	Numero	9
4.1.2	Carattere	9
4.1.3	Stringa	9
4.1.4	Simbolo	9
4.1.5	Lista	9
4.1.6	Array	10
4.1.7	Dizionario	10
4.2	Variabili	10
4.3	Costanti	11
4.4	Funzioni precostituite	11
4.4.1	Calcolo numerico	11
4.4.2	Comparazione	11
4.4.3	Manipolazione di stringhe	12
4.4.4	Manipolazione di array	12
4.4.5	Manipolazione di liste	12
4.4.6	Manipolazione di dizionari	13
4.4.7	Input/Output	13
4.4.8	Istruzioni condizionali	14
4.4.9	Cicli	14
4.5	Funzioni definite dall'utente	15
4.6	Scrivere il programma	15
5	Piccoli programmi di esempio	16
II	NewLISP	18
1	Installazione	18
2	Come funziona: il primo programma	18
3	Basi del linguaggio	19
4	Tipi fondamentali	20

5	Variabili e costanti	21
6	Funzioni preconfezionate	22
6.1	Input/Output	22
6.2	Aritmetica	23
6.3	Matematica	25
6.3.1	Matematica di base	25
6.3.2	Trigonometria	25
6.3.3	Statistica	25
6.3.4	Matematica finanziaria	27
6.3.5	Array e matrici	28
6.4	Manipolazione di liste	29
6.5	Manipolazione di stringhe	29
6.6	Manipolazione di simboli	29
6.7	Lavorare con i file	30
7	Costruire una funzione	32
8	Interattività con l'utente	34
9	Strutture di controllo	35
9.1	Esecuzione condizionale	35
9.2	Ripetizione	36

Parte I

Common LISP

1 Predisposizione del computer

In genere i linguaggi di programmazione dispongono di interpreti o compilatori in un unico esemplare, come accade per Python, Julia, Perl, Go, Java, Raku, oppure in più di un esemplare, comunque nel limite di poche unità, come avviene per C, C++ e Pascal.

Per Common LISP, in particolare per chi lavora su Linux, abbiamo invece l'imbarazzo della scelta.

Caratteristica comune è la presenza di un interprete che agisce su REPL (Read-Eval-Print-Loop), cioè in una shell dove si scrivono i comandi per vederli immediatamente eseguiti, e di un compilatore, generalmente dedicato alla conversione dei comandi in linguaggio macchina in modo da velocizzarne l'esecuzione.

Per lavorare con il LISP dobbiamo installare sul computer almeno un interprete/compilatore. In gergo si dice che dobbiamo avere sul computer una implementazione del Common LISP.

Qui descrivo le implementazioni più diffuse e di più facile acquisizione nel mondo del software libero, tutte gratuite.

1.1 CMUCL

E' la più anziana ed esiste per il sistema UNIX dal 1980, cioè da prima ancora che esistesse il Common LISP, col nome CMU e venne adattata al Common LISP nel 1985 assumendo il nome CMUCL, acronimo di Carnegie Mellon University Common Lisp.

Il suo compilatore si chiama Python, omonimia assolutamente casuale e che non ha nulla a che vedere con il linguaggio di programmazione Python che, ai tempi, nemmeno c'era.

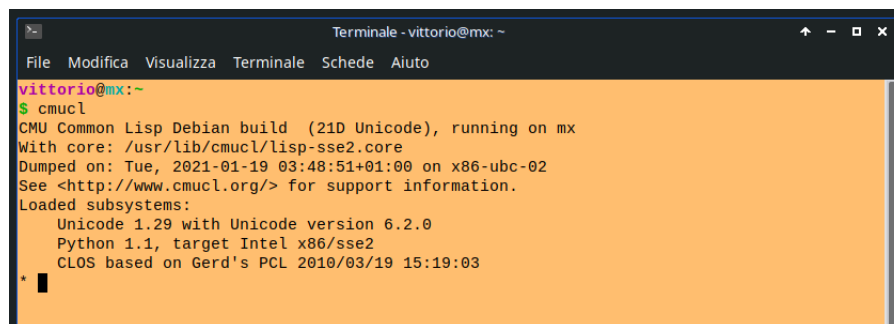
La troviamo all'indirizzo <https://www.cons.org/cmucl/>. Nella pagina del DOWNLOAD, aprendo la directory /RELEASE, troviamo i file di installazione per Linux e Mac.

Chi usa Linux quasi sicuramente trova ciò che serve nel repository della sua distro.

Probabilmente è l'implementazione di Common LISP che fornisce la performance più elevata.

Non è disponibile per Windows.

Una volta installata viene avviata con il comando a terminale `cmucl` e si presenta così



```
Terminale - vittorio@mx: ~
File Modifica Visualizza Terminale Schede Aiuto
vittorio@mx:~$ cmucl
CMU Common Lisp Debian build (21D Unicode), running on mx
With core: /usr/lib/cmucl/lisp-sse2.core
Dumped on: Tue, 2021-01-19 03:48:51+01:00 on x86-ubc-02
See <http://www.cmucl.org/> for support information.
Loaded subsystems:
Unicode 1.29 with Unicode version 6.2.0
Python 1.1, target Intel x86/sse2
CLOS based on Gerd's PCL 2010/03/19 15:19:03
* █
```

Per uscire si scrive il comando `(quit)`.

1.2 SBCL

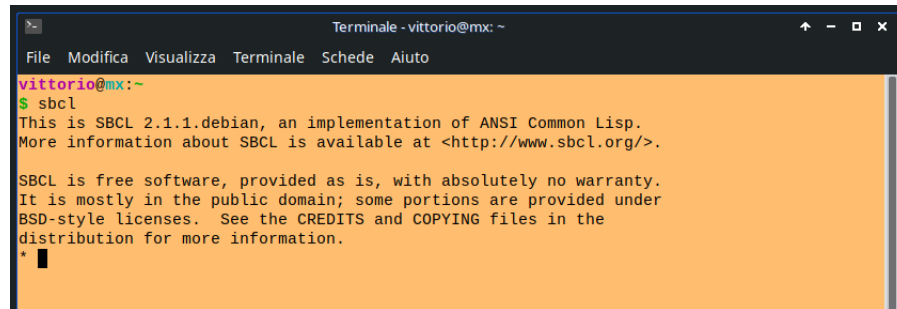
E' un fork di CMUCL e, dal momento che il Carnegie e il Mellon che danno il nome all'Università in cui questo è nato erano, rispettivamente, un re dell'acciaio e un grande banchiere, il suo nome è l'acronimo di Steel Bank Common Lisp.

Lo troviamo all'indirizzo <https://www.sbcl.org/>. Nella pagina del DOWNLOAD troviamo una comoda platform table con caselle corrispondenti alle varie versioni disponibili per vari sistemi operativi (oltre a Linux, anche Mac e Windows). Cliccando sulla casella corrispondente alla nostra situazione ci procuriamo quanto necessario per installare l'implementazione sul nostro computer.

Al solito, chi usa Linux molto probabilmente trova il tutto nel repository della sua distro.

Chi usa Windows, soprattutto se si tratta di un Windows datato, può trovare installer adatti anche all'indirizzo <https://github.com/akovalenko/sbcl-win32-threads/wiki>.

Una volta installata viene avviata con il comando a terminale `sbcl` e si presenta così



Per uscire si scrive il comando (`quit`).

1.3 GCL

Sta per GNU Common Lisp ed è l'implementazione di Common LISP nel progetto GNU.

Il suo compilatore ha la particolarità di produrre codice oggetto nativo generando innanzi tutto codice C e attivando poi un compilatore C.

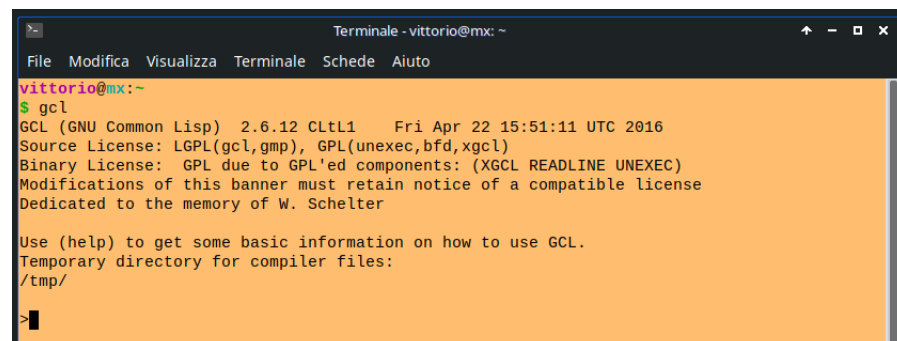
Questa particolarità lo rende adatto per lo sviluppo di grandi progetti impegnativi.

Si trova all'indirizzo <https://www.gnu.org/software/gcl/>.

Cliccando sulla voce di menu GET LATEST RELEASE apriamo una pagina in cui troviamo gli installer per Linux e Windows.

Se lavoriamo su Linux possiamo installare la versione presente nel repository della nostra distro, che viene in genere proposta nella sua intera vastità, comprendente anche il tool Tk per creare programmi dotati di interfaccia grafica utente.

Una volta installata viene avviata con il comando a terminale `gcl` e si presenta così



Per uscire si scrive il comando (`quit`).

1.4 CLISP

Altra implementazione che ci offre il progetto GNU, è il parente povero di GCL. Lo possiamo ritenere una versione di GNU Common Lisp per dilettanti.

E' presente nei repository delle distro Linux e possiamo usarla su Windows attraverso Cygwin MinGW/MSYS e installarlo su Mac attraverso MacPorts.

```

Terminale - vittorio@mx: ~
File Modifica Visualizza Terminale Schede Aiuto

vittorio@mx:~
$ clisp

      i i i i i i i
      I I I I I I I
      I \ \ + / / I
      \ \ + + / /
      - _ - | _ -
      |
      -----+-----

      00000 0      0000000 00000 00000
      8      8 8      8      8 o 8 8
      8      8      8      8      8
      8      8      8      00000 80000
      8      8      8      8 8
      8 o 8      8 o      8 8
      00000 8000000 0008000 00000 8

Welcome to GNU CLISP 2.49.92 (2018-02-18) <http://clisp.org/>

Copyright (c) Bruno Haible, Michael Stoll 1992-1993
Copyright (c) Bruno Haible, Marcus Daniels 1994-1997
Copyright (c) Bruno Haible, Pierpaolo Bernardi, Sam Steingold 1998
Copyright (c) Bruno Haible, Sam Steingold 1999-2000
Copyright (c) Sam Steingold, Bruno Haible 2001-2018

Type :h and hit Enter for context help.

[1]> █

```

* * *

Esistono molte altre implementazioni non altrettanto facili da trovare come le quattro che ho citato o non altrettanto alla portata anche di principianti (come alcune il cui compilatore traduce semplicemente il codice LISP in codice C per poi essere utilizzato in programmi compilati in C).

Ottima, forse migliore, alternativa per chi usa Linux ritengo sia CLISP.

Se usiamo l'implementazione SBCL, l'esecuzione del programma può avvenire, essendo posizionati nella directory dove abbiamo salvato il file del programma, scrivendo a terminale (quello che in Windows si chiama prompt dei comandi) il comando

6

`sbcl -- script <nome_file_programma>`².

Per il sistema operativo Linux possiamo scrivere nella prima riga del file di programma la shebang, in modo che il programma stesso possa essere eseguito semplicemente richiamando a terminale il file che lo contiene, previa abilitazione all'esecuzione con `chmod 555`.

La shebang per l'implementazione SBCL è

```
#! /usr/bin/sbcl --script3.
```

Altra possibilità di eseguire i nostri programmi è quella di caricarli nella REPL con la funzione `load`, usando questa sintassi

```
(load <file_programma>)
```

dove `<file_programma>` è, tra doppi apici, il percorso al file del programma.

Nel caso dell'implementazione GCL ciò funziona solo previa compilazione del file di programma.

Compilazione

Il primo, spesso unico, obiettivo della compilazione è quello di tradurre il nostro script di programma in linguaggio macchina in modo da poterlo eseguire più rapidamente.

Con la velocità che sviluppano i processori che troviamo oggi installati sui computer si tratta di una facilitazione di cui nemmeno ci accorgiamo, soprattutto se abbiamo a che fare con programmi da dilettanti.

La compilazione si può riferire ad una parte del programma, come una funzione o una macro che abbiamo nella REPL, con la sintassi

```
(compile <identificativo_funzione>).
```

In questo modo la compilazione avviene senza produrre alcun file e viene semplicemente ritenuta nella REPL in modo che la versione compilata sia disponibile per il successivo richiamo della funzione nella REPL stessa.

Possiamo compilare un intero file di programma con la sintassi

```
(compile-file <file_programma>)
```

dove `<file_programma>` è, tra doppi apici, il percorso al file del programma.

In questo caso, nella stessa directory dove si trova il file del programma viene prodotto il programma compilato, consistente, nel caso dell'implementazione SBCL, in un file con estensione `.fasl`⁴.

Il file compilato viene eseguito con le stesse modalità che abbiamo visto per il file di programma non compilato.

* * *

Abbiamo visto che tra le varie possibili implementazioni esistono alcune particolarità di funzionamento.

Tutte riconoscono tuttavia allo stesso modo il linguaggio Common LISP, le cui basi cerco di descrivere nelle pagine seguenti, in modo che anche un dilettante qualsiasi possa essere in grado di produrre qualche piccolo programma utilizzando questo linguaggio altamente professionale.

²Questa modalità non funziona nel caso dell'implementazione CMUCL. Nell'implementazione GCL il comando per eseguire un programma è `gcl -f`, seguito dal nome del file e nell'implementazione CLISP è `clisp`, seguito da nome del file.

³Questa modalità non funziona nel caso dell'implementazione CMUCL.

Nell'implementazione GCL la shebang è

```
#! /usr/lib/gcl-2.6.12/unixport/saved_ansi_gcl -f
```

e nel caso di CLISP è

```
#! /usr/bin/clisp
```

⁴Nell'implementazione CMUCL l'estensione del file è `.sse2f`, nell'implementazione GCL l'estensione del file è `.o`, trattandosi di un file oggetto utile per trattamento con linguaggio C e nell'implementazione CLISP l'estensione del file è `.fas`

3 Premessa al linguaggio

Il LISP è un linguaggio diverso da tutti gli altri.

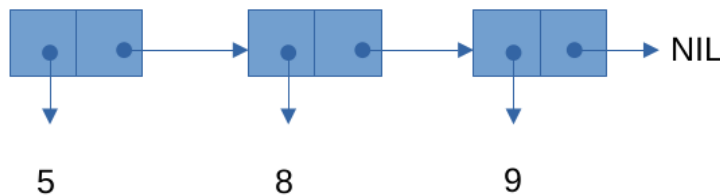
Questa diversità è dovuta al modo tutto originale con cui LISP utilizza la memoria del computer, attraverso il costrutto CONS.

Esso è composto da due metà.

Quella di sinistra, denominata CAR, contiene un puntatore al dato. Quella di destra, denominata CDR, contiene un puntatore al successivo CONS⁵.

Attraverso il puntatore al successivo CONS si crea la struttura di base del linguaggio LISP, la lista (ricordo che LISP sta per LISt Processing). Quando questo puntatore punta a nulla (NIL) la lista termina.

Graficamente possiamo esprimere tutto questo così



Questo costrutto nella memoria del computer viene espresso nel linguaggio LISP in questo modo

(5 8 9)

che è una lista che contiene gli elementi 5, 8 e 9.

Utilizzando il linguaggio LISP la possiamo costruire con l'istruzione

```
(cons 5 (cons 8 (cons 9 nil)))
```

o, più semplicemente ricorrendo alla funzione `list`, con l'istruzione

```
(list 5 8 9).
```

Il fatto che la lista sia costituita da un insieme di puntatori, tutto ciò che facciamo utilizzando la lista, che è la base di tutto ciò che facciamo in LISP, avviene con l'uso di puntatori.

La conseguenza più rilevante di questo è che il passaggio dei parametri alle funzioni del linguaggio avviene sempre per riferimento e mai per valore⁶.

La lista può contenere tra i suoi elementi altre liste. In tal modo si creano liste annidate e attraverso gli annidamenti si possono creare alberi.

Tutto questo strano impianto è finalizzato a poter elaborare con il linguaggio non solo dati numerici ma anche simboli, come richiesto da applicazioni che abbiano a che fare con l'intelligenza artificiale.

Ma si è così creato un linguaggio molto difficile da imparare e molto laborioso da utilizzare, soprattutto quando si voglia affrontare l'elaborazione simbolica.

Linguaggio difficile anche da leggere a causa della presenza di tutte le parentesi necessarie a definire le liste che compongono il programma, il che rende il linguaggio molto lontano dal modo umano di rappresentare operazioni, formule e funzioni, anche per l'uso della Reverse Polish Notation, che antepone gli operatori, in realtà funzioni, agli operandi. In compenso questo lo

⁵Anche le più strane espressioni hanno una spiegazione. CONS sta per Constructor. CAR e CDR derivano dalla terminologia relativa al primo computer su cui fu utilizzato il LISP, un IBM 704, i cui registri avevano più componenti, due dei quali erano chiamati Address Part e Decrement Part: da qui deriva CAR, che sta per Contents of the Address part of Register, e CDR, che sta per Contents of the Decrement part of Register.

⁶Nel linguaggio C, dove normalmente il passaggio dei parametri avviene per valore, per poter passare i parametri per riferimento si sono dovuti inventare i puntatori, che in LISP sono connaturati.

rende molto vicino al linguaggio macchina e semplifica la compilazione a vantaggio della velocità di elaborazione.

In questo manualletto si trova quanto serve per familiarizzare con le basi del linguaggio.

4 Il linguaggio

Il Common LISP è un linguaggio funzionale che riconosce determinati tipi di dato e li elabora attraverso funzioni precostituite nel linguaggio o sviluppabili dall'utente.

Per sapere tutto su questo linguaggio andiamo all'indirizzo <https://lisp-lang.org/>.

Per sapere il minimo che può servire per fare qualche cosa possiamo fermarci qui.

4.1 Tipi di dato

4.1.1 Numero

I valori numerici vengono trattati dal Common LISP come interi, frazioni, reali o complessi in funzione del contesto.

Intero Si scrive con uno o più caratteri numerici e viene letto e ritornato come tale.

Frazione Si scrive con caratteri numerici separati da / e viene letta e ritornata come tale.

Reale Si scrive con caratteri numerici utilizzando il punto (.) come separatore decimale e viene letto e ritornato come tale.

Complesso Si scrive avvalendosi della funzione `complex` così utilizzata:

(`complex` <parte_reale> <parte_immaginaria>)

e viene letto e ritornato come

#C(<parte_reale> <parte_immaginaria>).

Le singole parti sono estraibili rispettivamente con le funzioni `realpart` e `imagpart`.

4.1.2 Carattere

E' il carattere singolo. Si scrive con un carattere preceduto da #\ e viene letto e ritornato come tale.

Sono caratteri validi #\space e #\newline, ideati apposta per rendere visibili i caratteri spazio e accapo.

4.1.3 Stringa

E' una successione di caratteri scritta includendo la successione stessa tra doppi apici e viene letta e ritornata come tale. In realtà si tratta di un array di caratteri.

4.1.4 Simbolo

E' composto da uno o più caratteri alfanumerici scritti dopo un apice singolo (').

4.1.5 Lista

E' una successione di elementi separati da uno spazio racchiusa tra parentesi tonde e preceduta da apice singolo. Viene letta e ritornata come tale.

4.1.6 Array

Un array monodimensionale, cioè un vettore, si può creare utilizzando la funzione `vector` così:
(`vector` <primo_elemento> <secondo_elemento>)

Quello così costruito è un vettore a dimensione fissa e viene letto e ritornato tra parentesi tonde preceduto dal carattere #.

Più in generale l'array a dimensione fissa si predispone con la funzione `make-array` con la sintassi

(`make-array` '(<righe> <colonne> <altra_dimensione>)) e si può inizializzare l'array stesso attraverso l'opzione `:initial-contents` seguita da una lista contenente le liste rappresentanti ciascuna riga.

Con l'istruzione

```
(make-array '(2 2) :initial-contents '((3 4) (2 8)))
```

si crea la matrice

$$\begin{vmatrix} 3 & 4 \\ 2 & 8 \end{vmatrix}$$

L'array multidimensionale così creato viene letto e ritornato tra parentesi tonde preceduto dal carattere # e da un numero che indica la dimensione.

Nel caso dell'esempio viene ritornato

```
#2A((3 4) (2 8))
```

Per costruire un array a dimensione variabile occorre utilizzare l'opzione `:fill-pointer` seguita dall'indirizzo del punto da cui è possibile ampliare l'array.

4.1.7 Dizionario

E' una successione di elementi accoppiati a una parola chiave. E' detto anche array associativo o `plist` (property list).

E' racchiuso tra parentesi tonde precedute da apice singolo.

La parola chiave è preceduta da `:` e viene letta e ritornata come tale.

4.2 Variabili

La variabile è la posizione di memoria in cui viene allocato un dato di un tipo qualsiasi perché possa essere elaborato o cambiato, comunque tenuto a disposizione nel corso delle elaborazioni che deve fare il programma.

La variabile si definisce con la funzione `defvar` in questo modo

```
(defvar <nome_variabile>)
```

Se a <nome_variabile> si fa seguire un valore la variabile viene inizializzata.

Alla variabile si assegna un valore con la funzione `setq` in questo modo

```
(setq <nome_variabile> <valore>)
```

Se la variabile non è ancora stata definita essa viene qui definita e inizializzata. In certe implementazioni (come in SBCL) ciò genera un warning sul fatto che la variabile non era stata definita. In certe altre (come in CLISP) tutto fila senza problemi.

La variabile il cui nome è racchiuso tra i caratteri `*` è definita come variabile globale.

La variabile si tipizza automaticamente a seconda del tipo di valore che le viene assegnato.

4.3 Costanti

La costante è la posizione di memoria in cui viene allocato un dato di un tipo qualsiasi perché possa essere tenuto a disposizione, invariato, nel corso delle elaborazioni che deve fare il programma.

La costante si definisce e si inizializza con la funzione `defconstant` in questo modo
(`defconstant` <nome_costante> <valore>)

La costante il cui nome è racchiuso tra i caratteri `*` è definita come costante globale.

4.4 Funzioni precostituite

Qualsiasi cosa facciamo con il linguaggio LISP la facciamo utilizzando una funzione. Persino la somma tra due numeri, che altrove eseguiamo utilizzando l'operatore `+`, qui la facciamo utilizzando la funzione `+`.

Il Common LISP ci offre decine e decine di funzioni preconfezionate per fare tantissime cose e ci offre pure il modo di creare noi altre funzioni, e vedremo come.

Qui richiamo le funzioni preconfezionate di più ricorrente uso.

4.4.1 Calcolo numerico

<code>+</code>	addizione
<code>-</code>	sottrazione
<code>*</code>	moltiplicazione
<code>/</code>	divisione (tra interi resto 0 o tra reali)
<code>mod</code>	resto di divisione
<code>truncate</code>	parte intera di un numero reale
<code>round</code>	arrotondamento di un reale all'intero più vicino
<code>floor</code>	arrotondamento di un reale per difetto
<code>ceiling</code>	arrotondamento di un reale per eccesso
<code>abs</code>	valore assoluto
<code>sqrt</code>	radice quadrata
<code>expt</code> <base> <esponente>	elevamento a potenza
<code>log</code> <numero> <base>	logaritmo (se la base non è indicata logaritmo naturale)
<code>sin</code> <code>cos</code> <code>tan</code>	funzioni trigonometriche per angolo in radianti
<code>asin</code> <code>acos</code> <code>atan</code>	funzioni trigonometriche inverse
<code>sinh</code> <code>cosh</code> <code>tanh</code>	funzioni iperboliche
<code>asinh</code> <code>acosh</code> <code>atanh</code>	funzioni iperboliche inverse
<code>min</code>	valore minimo di una successione di numeri
<code>max</code>	valore massimo di una successione di numeri
<code>lcm</code>	minimo comune multiplo di una successione di numeri
<code>gcd</code>	massimo comun divisore di una successione di numeri

Per agevolare determinati calcoli il numero pi greco è memorizzato nella costante di sistema `PI`.

4.4.2 Comparazione

<code>=</code>	uguale
<code>/=</code>	diverso
<code><</code>	minore
<code>></code>	maggiore
<code><=</code>	minore o uguale
<code>>=</code>	maggiore o uguale

I simboli indicati valgono per la comparazione tra valori numerici.

Se al simbolo si fa precedere `char` o `string` senza spazi intermedi (esempio `char=` o `string<`) la comparazione vale, rispettivamente, per caratteri e stringhe.

Esclusivi per la comparazione tra valori numerici sono

`zerop` uguale a zero
`minusp` minore di zero
`plusp` maggiore di zero

I valori booleani restituiti dalle funzioni di comparazione sono `T` per vero e `NIL` per falso.

4.4.3 Manipolazione di stringhe

`concatenate 'string <stringa> <stringa> . . .` concatena stringhe.

`char <stringa> <posizione>`

estrae il carattere che si trova in una certa posizione nella stringa, con indice che parte da zero sul primo carattere a sinistra.

4.4.4 Manipolazione di array

Per lavorare su un array è bene innanzi tutto crearne un'istanza ed assegnarla ad una variabile con (`setq <nome_variabile> (make-array '(<righe> <colonne> . . .))`)

Come abbiamo visto nel paragrafo 4.1.6, attraverso l'opzione `:initial-contents` possiamo anche inizializzare l'array che andiamo a creare. Se non utilizziamo questa opzione creiamo un array delle dimensioni indicate e con elementi uguali a 0.

Dopo di che

`setf (aref <nome_variabile> <riga> <colonna> . . .) <valore>`
assegna i valori nella posizione indicata

`aref <nome_variabile> <riga> <colonna> . . .`
estrae il valore contenuto nella posizione indicata

`array-rank <nome_variabile>`
indica l'ordine dell'array

Nel caso di array monodimensionali (vettori) abbiamo le semplificazioni

(`setq <nome_variabile> (vector <elemento> <elemento> . . .)`)

per creare l'istanza del vettore, inizializzarlo ed assegnarlo ad una variabile

`svref <nome_variabile> <posizione>`

per estrarre il valore contenuto nella posizione indicata

Rammento che tutti gli indici per indicare la posizione partono da 0

4.4.5 Manipolazione di liste

`cons <elemento> <lista>`

crea una lista aggiungendo un primo elemento a quelli di un'altra lista

`nth <indice> <lista>`

estrae da una lista l'elemento corrispondente all'indice (partendo da zero)

in luogo di `nth <indice>` si può usare `first`, `second`, ecc.

`append <lista> <lista> . . .`

concatena liste

`union <lista> <lista>`

determina l'unione di due liste (in senso insiemistico)

`intersection <lista> <lista>`

determina l'intersezione di due liste (in senso insiemistico)

`length <lista>` ritorna il numero degli elementi di una lista
`remove <elemento> <lista>`
crea una lista con gli elementi di un'altra lista rimuovendo quello indicato

4.4.6 Manipolazione di dizionari

Serve bene un esempio.

Creiamo il dizionario chiamato Capitali:
(setq Capitali '(:Italia Roma :Francia Parigi))
Con
(getf Capitali :Francia)
chiediamo e ci viene indicata la capitale della Francia
Con
(setq Capitali (append Capitali '(:Germania Berlino)))
arricchiamo il dizionario aggiungendo la capitale della Germania.

4.4.7 Input/Output

Per l'input di dati dalla tastiera abbiamo a disposizione due funzioni:

`read` legge un numero o un carattere dalla tastiera,
`read-line` legge una stringa, anche con spaziature, dalla tastiera.

Anche per l'output abbiamo due funzioni:

`print` per scrivere sullo schermo in maniera grezza,
`format` per scrivere sullo schermo in modo formattato, con la sintassi

`format t "<stringa> <segnaposto>" <valore_per_segnaposto>`

dove il `<valore_per_segnaposto>` può essere indicato richiamando il nome della variabile che lo contiene, essere scritto nella forma prevista dal suo tipo o anche attraverso una espressione per calcolarlo.

Il segnaposto multiuso, se non si hanno particolari esigenze di formattazione di dati numerici è
~a ⁷.

Altro segnaposto molto utile è

~% che si usa, senza poi indicarne il valore, per andare a capo (sta per newline).

Per la formattazione di numeri interi abbiamo a disposizione le funzioni:

~d per esprimere l'intero in base decimale
~x per esprimere l'intero in base esadecimale
~o per esprimere l'intero in base ottale
~b per esprimere l'intero in formato binario
~nr per esprimere l'intero in base qualsiasi indicata da n
~@r per esprimere l'intero in numerazione romana
~r per esprimere l'intero in lettere inglesi
~:r per esprimere l'intero, come ordinale, in lettere inglesi

Nei primi 5 casi, possiamo inserire tra la ~ e la sigla del segnaposto l'ampiezza, in numero di caratteri, dello spazio in cui posizionare il numero con allineamento a destra.

Questo è un esempio di istruzione con relativo risultato

⁷Rammento che il carattere ~ (detto tilde), con la tastiera Linux si ottiene premendo insieme i tasti Alt Gr e ì, con la tastiera Mac premendo insieme i tasti Alt e 5 e con la tastiera Windows premendo insieme Alt e 126 della tastierina numerica

```
* (format t "~15d~%~15d" 16 1472)
      16
     1472
```

Per la formattazione di numeri a virgola mobile usiamo il segnaposto

~f

e, appena dopo la ~, possiamo inserire, in numero di caratteri, l'ampiezza dello spazio in cui posizionare il numero e, separato da virgola, il numero di cifre decimali da evidenziare.

In questo esempio viene stampato il valore della costante di sistema π con tre cifre decimali

```
* (format t "~~,3f" PI)
3.142
```

In quest'altro vengono posizionati i numeri con allineamento a destra e due cifre decimali

```
* (format t "~10,2f~%~10,2f" 12.75648 1654.762435)
      12.76
     1654.76
```

4.4.8 Istruzioni condizionali

Se le istruzioni stanno in una sola espressione:

```
if (<condizione>) <istruzione>
```

viene eseguita <istruzione> se <condizione> è vera

o anche

```
if (<condizione>) <istruzione_per_vero> <istruzione_per_falso>
```

Se le istruzioni sono complesse e richiedono l'uso di più espressioni:

```
if (<condizione>) (progn <istruzione> <istruzione> ...)
```

o, meglio:

```
when (<condizione>) <istruzione> <istruzione> ...
```

In presenza della necessità di elaborare condizioni multiple si può evitare di annidare gli if usando

```
cond ((<condizione>) <istruzione> (<condizione>) <istruzione> ...)
```

Nel caso <condizione> richieda la contemporanea verifica di più situazioni si utilizzano gli operatori logici booleani

and (e logico)

or (o logico)

not (negazione logica)

4.4.9 Cicli

Per eseguire un'istruzione un numero definito di volte abbiamo a disposizione, tra le tante, queste due funzioni semplici semplici:

```
loop repeat <n> do <istruzione>
```

```
dotimes (<contatore> <n>) <istruzione>
```

Per esempio, per scrivere 4 volte la stringa "Ciao" possiamo utilizzare

```
(loop repeat 4 do (print "Ciao"))
```

oppure

```
(dotimes (i 4) (print "Ciao"))
```

Per reiterare operazioni sugli elementi di una lista, tra le tante disponibili, abbiamo la facile funzione

dolist (<variabile> <lista>) <istruzione>

Per esempio

```
(dolist (x '(2 3 4)) (print (* x x)))
```

scrive i numeri 4 9 e 16.

4.5 Funzioni definite dall'utente

Possiamo noi stessi definire una funzione con la sintassi

```
(defun <nome_funzione> (<parametro> <parametro> ...) (<espressione>))
```

In questo modo, per esempio, possiamo definire una funzione per il calcolo del fattoriale di un numero

```
(defun fattoriale (n) (if (zerop n) 1 (* n (fattoriale (- n 1)))))
```

La definizione di una funzione può servire quando in un programma si debba fare più volte uno stesso calcolo e si voglia evitare di riscrivere ogni volta la formula per eseguirlo. Una volta definita la funzione, infatti, basta richiamarla con il suo nome fornendo i parametri richiesti.

Possiamo anche salvare la nostra funzione in un file con estensione .lsp e caricarla nella REPL con

```
(load <percorso_al_file>)
```

in modo da poterla utilizzare nella REPL stessa richiamandola con il nome e fornendo i parametri richiesti.

Possiamo anche definire temporaneamente una funzione per utilizzarla al volo attraverso la così detta espressione lambda, con la seguente sintassi

```
(lambda (<parametro> <parametro> ...) (<espressione>))
```

Per esempio, se nella REPL scriviamo

```
((lambda (x) (* x x x)) 3)
```

otteniamo il risultato 27.

4.6 Scrivere il programma

Le istruzioni che compongono il programma nel linguaggio LISP vengono eseguite attraverso la così detta valutazione di liste.

Ciascuna istruzione che debba essere eseguita deve essere contenuta in una lista.

Si tratta però di una lista diversa da quella che abbiamo visto nel paragrafo 4.1.5. Quella è un tipo contenitore di dati e consiste in una serie di elementi contenuti tra parentesi tonde preceduta da apice singolo.

L'interprete LISP non valuta una lista preceduta da apice singolo ma la ritorna tale e quale.

La lista non preceduta da apice viene valutata e perché la valutazione possa avvenire occorre che il primo elemento sia una funzione. Gli altri elementi che seguono sono i parametri che vengono passati alla funzione stessa.

La semplice somma di due numeri avviene attraverso questo meccanismo.

Se scriviamo nella REPL

```
'(+ 4 5)
```

ci viene restituita la lista (+ 4 5)

Abbiamo, infatti, una lista che, essendo preceduta da apice non viene valutata.

Se scriviamo nella REPL

```
(+ 4 5)
```

ci viene restituito il valore 9

Abbiamo, infatti, una lista che, non essendo preceduta da apice viene valutata: essa contiene come primo elemento la funzione + seguita dai parametri 4 e 5 e la valutazione fornisce il risultato 9.

Se scriviamo nella REPL

```
(3 4 5)
```

viene segnalato errore

Abbiamo, infatti, una lista che, non essendo preceduta da apice viene valutata ma l'interprete, non trovando la funzione oggetto di valutazione, segnala errore.

Possiamo anteporre ad una lista un apice rovesciato (il carattere ```)⁸ ottenendo di evitare la valutazione della lista potendo tuttavia contrassegnare all'interno di questa una lista da valutare anteponendovi una virgola (il carattere `,`).

Ad esempio, se scriviamo nella REPL

```
`(4 + 5 = , (+ 4 5))
```

 ci viene restituito `(4 + 5 = 9)`

in quanto l'interprete, fino alla virgola legge semplicemente `(4 + 5 =` e, dopo la virgola valorizza la lista `(+ 4 5)` e restituisce il risultato dell'addizione.

Un programma contiene una o più, tantissime nei programmi più complessi, liste da valorizzare e sono valorizzate dall'interprete una dopo l'altra, nell'ordine in cui si presentano.

Abbiamo già visto nel Capitolo 2 in che modo scrivere ed eseguire i programmi.

Quanto allo scrivere, rammento che LISP è case insensitive, cioè non distingue tra maiuscolo e minuscolo: la funzione `print` può essere indifferentemente essere richiamata con `print`, `PRINT`, `Print` o `PrInT`.

5 Piccoli programmi di esempio

Con il programma, meglio chiamarlo script, riportato all'inizio della pagina seguente viene chiesto il nome all'utente per rivolgergli un saluto.

La prima riga è la shebang per l'implementazione SBCL, che, nel sistema Linux, consente di eseguire il programma, una volta reso eseguibile, senza richiamare l'interprete ma richiamando semplicemente il nome del file che lo contiene. Fuori dal sistema operativo Linux questa riga viene semplicemente ignorata.

```
#!/usr/bin/sbcl --script
(format t "Come ti chiami?~%")
(defvar nome)
(setq nome (read-line))
(format t "Ciao, ~a!~%" nome)
```

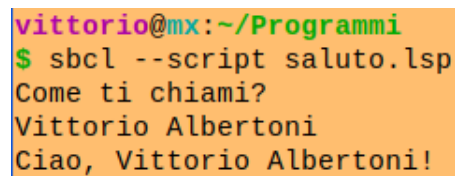
La seconda riga formula all'utente, in testo formattato, la richiesta del suo nome.

La terza riga inizializza la variabile in cui memorizzare il nome dell'utente. Potremmo evitare questa istruzione, ma gli interpreti di talune implementazioni, come la SBCL, eseguirebbero il programma propinandoci un warning al momento di eseguire l'istruzione successiva, in quanto la variabile `nome` è sconosciuta. Il programma però funzionerebbe comunque.

La quarta riga memorizza nella variabile `nome` quanto inserito dalla tastiera.

La quinta riga formula il saluto, in testo formattato, destinandolo al nome memorizzato nella variabile `nome`.

Memorizzato il programma nel file `saluto.lsp` può essere eseguito a terminale con questo risultato



```
vittorio@mx:~/Programmi
$ sbcl --script saluto.lsp
Come ti chiami?
Vittorio Albertoni
Ciao, Vittorio Albertoni!
```

⁸L'apice rovesciato o apice inverso o backtick si ottiene su tastiera Linux premendo contemporaneamente i tasti `AltGr` e ```, su tastiera Mac premendo contemporaneamente i tasti `Alt` e `9` e su tastiera Windows premendo contemporaneamente i tasti `Alt` e `96` della tastierina numerica.

Quest'altro script calcola il fattoriale del numero indicato dall'utente

```
#! /usr/bin/sbcl --script
(defun fattoriale(n) (if(zerop n)1 (* n (fattoriale(- n 1)))))
(format t "Inserisci il numero di cui vuoi calcolare il fattoriale~%")
(defvar numero)
(setq numero (read))
(format t "Il fattoriale di ~a è~%~a~%" numero (fattoriale numero))
```

Dopo la riga dedicata alla shebang abbiamo la definizione di una funzione per calcolare il fattoriale di un numero. Come si vede si tratta di una bella funzione ricorsiva (funzione che utilizza sé stessa), a dimostrazione del fatto che LISP supporta la ricorsività.

Seguono le righe per richiedere all'utente il numero di cui calcolare il fattoriale, per definire la variabile in cui collocarlo e per collocarvelo leggendolo dalla tastiera.

Infine, nell'ultima riga, la formattazione del testo per evidenziare il risultato, calcolato richiamando la funzione prima creata e passandole il relativo parametro richiamando il nome della variabile in cui è memorizzato.

Memorizzato il programma nel file `fattoriale.lsp` può essere eseguito a terminale con questo risultato

```
vittorio@mx:~/Programmi  
$ sbcl --script fattoriale.lsp  
Inserisci il numero di cui vuoi calcolare il fattoriale  
102  
Il fattoriale di 102 è  
961446671503512660926865558697259548455355905059659464369444714048531715130254590603314961  
882364451384985595980362059157503710042865532928000000000000000000000000
```

Notiamo come, pur trattandosi di un numero enorme, esso venga scritto nella sua interezza, addirittura andando a capo dopo aver occupato i 90 caratteri ospitabili in una riga del terminale. Ciò è dovuto al fatto che in LISP i numeri interi vengono trattati in precisione arbitraria, cioè senza alcuna limitazione imposta dal software ma con l'unica limitazione posta dall'hardware, che, oggi come oggi, può reggere numeri interi fino ad alcune migliaia di caratteri.

Ora un piccolo script che traduce un numero intero in numero secondo la numerazione usata nell'antica Roma.

```
#! /usr/bin/sbcl --script
(format t "Inserisci il numero da trascrivere in numerazione romana~%" )
(defvar numero)
(setq numero (read))
(format t "Il numero ~a era scritto così dagli antichi romani~%~@r~%" numero numero)
```

Ormai penso che i commenti siano superflui.

Memorizzato il programma nel file `romano.lsp` può essere eseguito a terminale con questo risultato

```
vittorio@mx:~/Programmi
$ sbcl --script romano.lsp
Inserisci il numero da trascrivere in numerazione romana
1745
Il numero 1745 era scritto così dagli antichi romani
MDCCXLV
```

Parte II

NewLISP

1 Installazione

Dopo il pensionamento del patron Lutz Müller, NewLISP si trova all'indirizzo <https://sourceforge.net/projects/newlisp/>.

Se apriamo la scheda FILES abbiamo modo di scaricare l'eseguibile per Windows e il source.

Windows

Per l'installazione su Windows dobbiamo lanciare l'installatore.

Mac OS X

Per l'installazione su Mac andiamo all'indirizzo <https://formulae.brew.sh/formula/newlisp>.

Linux

Sul nostro sistema Linux potrebbe essere già installato NewLISP: conviene pertanto verificarlo, ad evitare lavori inutili, scrivendo a terminale il comando `newlisp`.

Sempre su Linux l'installazione potrebbe avvenire attraverso il gestore pacchetti.

Il sistema più laborioso è quello della compilazione del source, che è contenuto nel file `newlisp-xx.x.x.tgz`.

Una volta scaricato questo file, lo copiamo, con poteri di root, in `/usr/local/src`.

Lo scompattiamo con `tar xzvf newlisp-xx.x.x.tgz`.

Ci posizioniamo nella directory che si è così creata a fianco, `newlisp_xx.x.x`, e diamo il comando `./configure`.

Indi diamo, uno dopo l'altro, i comandi `make` e `sudo make install`.

Se tutto ha funzionato possiamo eliminare il contenuto della directory `/usr/local/src`.

All'indirizzo

https://web.mit.edu/newlisp_v10.7.1/share/doc/newlisp/newlisp_manual.html
troviamo un ricco manuale in inglese.

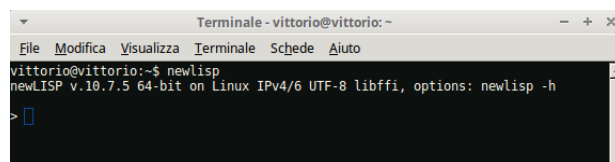
2 Come funziona: il primo programma

Una volta installato NewLISP possiamo usare il comando `newlisp` in tre modi:

- . scriverlo a terminale da solo,
- . scriverlo a terminale seguito dal nome di un file con estensione `.lsp`,
- . scriverlo a terminale con sintassi `atta` a «compilare» un file con estensione `.lsp`.

Nel primo caso si apre nel terminale una shell interattiva in cui possiamo utilizzare il linguaggio NewLISP per esperimento o per ottenere risultati di meno difficoltoso raggiungimento.

Questa è la shell nel mio terminale Linux



```
Terminale - vittorio@vittorio: ~  
File Modifica Visualizza Terminale Schede Aiuto  
vittorio@vittorio:~$ newlisp  
newLISP v.10.7.5 64-bit on Linux IPv4/6 UTF-8 libffi, options: newlisp -h  
> █
```

Se in corrispondenza del prompt della shell (simbolo >) scriviamo la seguente istruzione NewLISP
(println "Ciao mondo!")

nella riga sottostante compare immediatamente il saluto.

Normalmente la shell accetta una sola riga di comando.

Per scrivere un comando su più righe occorre racchiudere queste righe tra un tag di apertura [cmd] e un tag di chiusura [/cmd].

Nel secondo caso, previa scrittura in un editor di testo di queste istruzioni

```
(println "Ciao mondo!")
```

```
(exit)
```

e memorizzazione in un file con estensione .lsp, per esempio ciao_mondo.lsp, scrivendo a terminale

```
newlisp ciao_mondo.lsp
```

sempre a terminale vedremo comparire il saluto.

Nel terzo caso, con l'istruzione

```
newlisp -x ciao_mondo.lsp ciao_mondo
```

oppure, se siamo in Windows,

```
newlisp -x ciao_mondo.lsp ciao_mondo.exe
```

otteniamo la «compilazione» del nostro programmino.

Scrivo «compilazione» tra virgolette in quanto non si tratta di una vera e propria compilazione ma della produzione di un file che contiene l'interprete: il risultato è comunque quello di poter distribuire il programma e poterlo eseguire anche su un computer su cui non è installato NewLISP. Se siamo su Linux dobbiamo preventivamente rendere eseguibile il file con il comando

```
chmod 555 ciao_mondo
```

o comando similare.

Abbiamo così costruito il nostro primo programma in NewLISP.

3 Basi del linguaggio

La base del linguaggio LISP (LIST Processing) è la lista.

Una lista è una sequenza di elementi separati da uno spazio e racchiusa tra parentesi tonde.

Normalmente il primo elemento della lista viene visto da NewLISP come una funzione e gli elementi successivi vengono visti come parametri necessari alla funzione per raggiungere il risultato richiesto.

Il nostro primo programmino (println "Ciao mondo!") è una lista di due elementi: il primo è la funzione predefinita nel linguaggio per scrivere una cosa e andare a capo (println), il secondo è una stringa che contiene ciò che va scritto.

In NewLISP, come in tutti gli altri LISP che esistono, tutto viene eseguito attraverso funzioni che devono occupare il primo posto di una lista.

La stessa semplicissima somma di due numeri interi, che da qualsiasi altra parte viene effettuata attraverso l'operatore + inserito tra i due numeri da sommare, in NewLISP (in LISP non esistono gli operatori) viene effettuata richiamando la funzione +, che è la funzione preposta per sommare numeri con risultato intero, come primo elemento di una lista seguito dai numeri da sommare.

Sicché la somma di 4 e 5 si ottiene con l'istruzione (+ 4 5).

La trasformazione della lista in risultato avviene immediatamente in quanto la lista viene immediatamente valutata.

Per evitare questo possiamo anteporre alla lista un semplice apice ('), altrimenti detto apostrofo (codice ASCII 39).

Se nella shell interattiva scriviamo (+ 4 5) otteniamo il risultato 9.

Se scriviamo '(+ 4 5) otteniamo (+ 4 5).

In questo modo abbiamo l'opportunità di utilizzare la lista non come istruzione di programma ma come contenitore di dati.

4 Tipi fondamentali

I tipi di dati manipolabili con NewLISP sono i seguenti.

Numero

I valori numerici vengono trattati come numeri interi (`int`) e numeri in virgola mobile (`float`).

L'intero si scrive con uno o più caratteri numerici e viene letto e ritornato come tale.

Nella shell NewLISP a terminale, `5` ritorna `5`.

Il numero in virgola mobile si scrive con caratteri numerici utilizzando la virgola come separatore decimale e viene letto e ritornato come tale.

`5,6` ritorna `5,6`.

Contrariamente a quanto avviene in altri linguaggi di programmazione, dove impera il punto come separatore decimale, NewLISP, se siamo in Italia ed abbiamo un sistema operativo in lingua italiana, capisce le nostre abitudini e si aspetta che usiamo la virgola.

Possiamo verificarlo dando nella shell il comando (`div 3 2`) e vedere se il risultato viene scritto `1,5` o `1.5`.

Si può trasformare un `float` in `int` con l'istruzione (`int <numero_float>`). Ciò comporta la sparizione della parte decimale senza arrotondamento:

`(int 5,6)` ritorna `5`.

Si può trasformare in numero una stringa contenente caratteri numerici con le istruzioni (`int <stringa>`) o (`float <stringa>`):

`(int "5,6")` ritorna `5`,

`(float "5,6")` ritorna `5,6`.

Contrariamente si può anche trasformare un numero in una stringa con l'istruzione (`string <numero>`):

`(string 5,6)` ritorna `"5,6"`.

Se la stringa contiene caratteri che identificano un numero in base diversa da 10, con la funzione `int` possiamo tradurlo nel corrispondente numero intero in base 10, antepoendo nella stringa il prefisso `0x` se si tratta di un numero esadecimale, il prefisso `0` se si tratta di un numero ottale e il prefisso `0b` se si tratta di un numero binario.

Il numero esadecimale `5E7A` si converte in decimale con (`int "0x5E7A"`) che ritorna `24186`.

Il numero ottale `72` si converte in decimale con (`int "072"`) che ritorna `58`.

Il numero binario `11101` si converte in decimale con (`int "0b11101"`) che ritorna `29`.

Simbolo

Qualsiasi carattere o combinazione di caratteri che non siano numerici è un simbolo.

Il simbolo preceduto da un semplice apice (`'`) viene letto e ritornato come tale.

`'a` ritorna `a`,

`'abc` ritorna `abc`.

Stringa

La stringa è una successione di caratteri inclusa tra doppi apici e viene letta e ritornata come tale.

`"Vittorio"` ritorna `"Vittorio"`.

Lista

La lista, quella preceduta da apice che abbiamo visto prima, detta *quoted list*, è una successione di elementi di qualsiasi tipo.

'(abc g x z) genera una lista di simboli,
'(2 5 7 9) genera una lista di numeri,
'("Pippo" "uccello" "Sabato") genera una lista di stringhe,
'(sin sqrt add) genera una lista di funzioni,
'(35,8 sqrt "Giuseppe" abc) genera una lista mista.

Array

L'array, pur avendo maggior utilità pratica se composto da numeri, è un modo di organizzare dati di qualsiasi tipo.

L'array monodimensionale, detto altrimenti vettore, è molto simile alla lista.

L'array multidimensionale è una matrice.

L'array si genera con la funzione `array`, che ha la seguente sintassi.

(array 3 '(1 3 6)) genera il vettore (1 3 6),

(array 2 2 '(3 4 1 2)) genera la matrice ((3 4) (1 2)), altro modo di scrivere $\begin{vmatrix} 3 & 4 \\ 1 & 2 \end{vmatrix}$.

5 Variabili e costanti

Una variabile è un indirizzo di memoria ove si trova un dato e si identifica con un simbolo.

La funzione per creare una variabile è `set` con la seguente sintassi:

(set <simbolo> <dato>)

ove <dato> può essere una lista che genera un dato.

Con (set 'x 12,5) creiamo la variabile `x` che contiene il valore numerico 12,5.

Con (set 'a (+ 4 3)) creiamo la variabile `a` che contiene la somma di 4 e 3, cioè il valore numerico 7.

Con (set 'nome "Vittorio") creiamo la variabile `nome` che contiene la stringa "Vittorio".

Con (set 'M (array 2 2 '(3 4 1 2))) creiamo la variabile `M` che contiene la matrice $\begin{vmatrix} 3 & 4 \\ 1 & 2 \end{vmatrix}$.

Con (set 'l '(4 5 ab)) creiamo la variabile `l` che contiene la lista (4 5 ab).

La variabile creata con `set` è una variabile globale, cioè una variabile utilizzabile nel corso di un intero programma in più liste.

Se desideriamo creare, all'interno di una lista, una variabile locale che svolga la sua funzione solo all'interno della lista e che scompaia una volta che la lista ha operato possiamo utilizzare la funzione `let`, con la seguente sintassi:

(let (<simbolo> <dato>) <altre_istruzioni>)

ove <dato> può essere una lista che genera un dato e <altre_istruzioni> è il restante contenuto della lista.

Con (let (nome "Vittorio") (println "Ciao " nome)) scriviamo un saluto utilizzando la variabile `nome` creata come variabile locale all'interno della lista. Una volta che la lista ha esaurito il proprio compito scrivendo il saluto la variabile `nome` viene eliminata.

Il valore di una variabile si può modificare nel corso del programma.

Se vogliamo avere a disposizione dati utili per sviluppare il nostro programma ma che non siano modificabili, conviene che creiamo delle costanti che li contengano. La sintassi è

(constant <simbolo> <dato>)

ove <dato> può essere una lista che genera un dato.

Con (`constant 'c 8`) creiamo la costante `c` che contiene il numero 8.

Con (`constant 'e (exp 1)`) creiamo la costante `e` che contiene il numero e , base dei logaritmi naturali (2,718281828459045).

6 Funzioni preconfezionate

Tutto ciò che si fa con NewLISP lo si fa utilizzando funzioni: abbiamo visto che la lista normale (quella senza l'apostrofo) deve contenere come primo elemento una funzione e gli altri elementi sono i dati di cui questa funzione ha bisogno per produrre un risultato.

NewLISP ci mette a disposizione tantissime funzioni preconfezionate, utilizzabili per creare i nostri programmi: esse sono elencate nel manuale di NewLISP, che possiamo consultare o scaricare da <http://www.newlisp.org/>. Le troviamo nella sezione Function Reference, organizzate per gruppi o in ordine alfabetico. Per ciascuna funzione preconfezionata possiamo così conoscere a cosa serve, come si richiama, con quali parametri e di che tipo.

Qui richiamo dettagliatamente solo quelle che è necessario conoscere per le funzionalità di base o poco più.

6.1 Input/Output

`read-line`

legge l'input dal device corrente (per default la tastiera) come stringa delimitata dal carattere di fine riga.

Richiamata questa funzione il computer si mette in attesa che l'utente digiti sulla tastiera ciò che vuole inserire e preme INVIO.

Dal momento che quanto digitato viene acquisito come stringa, se ciò che interessa acquisire è un numero occorre fare la trasformazione.

Se rispondiamo a (`read-line`) digitando la parola ciao il computer legge la stringa "ciao".

Se rispondiamo digitando 16,25 il computer legge la stringa "16,25".

Se usiamo il costrutto (`float(read-line)`) e rispondiamo digitando 16,25 il computer legge il numero 16,25.

In genere questa funzione si utilizza per memorizzare il dato acquisito in una variabile, nel qual caso si utilizza la sintassi

(`set <nome_variabile> (read-line)`) se si vuole inserire nella variabile una stringa,

(`set <nome_variabile> (int(read-line))`) se si vuole inserire nella variabile un numero intero,

(`set <nome_variabile> (float(read-line))`) se si vuole inserire nella variabile un numero in virgola mobile,

dove `<nome_variabile>` è il simbolo (una o più lettere precedute a apostrofo ') con cui intendiamo nominare la variabile.

`println`

scrive sul device corrente (per default lo schermo) andando a capo dopo la scrittura,

`print`

scrive senza andare a capo dopo la scrittura.

Il parametro di questa funzione (ciò che deve essere scritto) si può indicare creandolo al momento con un'istruzione o richiamando una variabile che contiene quanto si vuole scrivere.

Esempi:

(`println "ciao"`) scrive la parola ciao,

(`println (* 3 2)`) scrive il numero 6 (risultato della moltiplicazione tra 3 e 2),

(`println 5,28`) scrive il numero 5,28,

(`println x`) scrive il contenuto della variabile `x`.

Quando si scrivono numeri a volte si sente l'esigenza di formattarli, o per allinearli o per fissare il numero di decimali. Possiamo farlo applicando la funzione `format` al numero da scrivere, indicato come tale, con un'espressione che lo genera o richiamando la variabile che lo contiene.

La sintassi della funzione `format` è

```
(format {<direttiva_di_formattazione>} <numero>)
```

dove <direttiva_di_formattazione> è

```
%<spazi_allineamento>.<decimali>f
```

con <spazi_allineamento> ad indicare il numero di spazi entro cui fissare l'allineamento a destra: se il numero indicato è inferiore al numero dei caratteri da scrivere l'allineamento avviene a sinistra, per ottenere il quale basta indicare 1,

e <decimali> ad indicare il numero di cifre decimali da scrivere.

Esempi:

```
(println (format {%1.3f} 7,895746))
```

 scrive 7,896 allineato a sinistra,

```
(println (format {%10.2f} (div 2 3)))
```

 scrive 0,67 allineato a destra in uno spazio di 10 caratteri.

6.2 Aritmetica

Abbiamo innanzi tutto funzioni che operano su numeri interi. Se passiamo a queste funzioni numeri in virgola mobile esse operano solo sulla parte intera dei numeri e ritornano come risultato un numero intero, senza approssimazioni di sorta: le parti decimali vengono semplicemente ignorate, come se non ci fossero. Si tratta delle seguenti funzioni:

+ somma tutti i numeri inseriti nella lista,

- sottrae al primo numero inserito il secondo e via via tutti gli altri inseriti,

* moltiplica tra loro tutti i numeri inseriti nella lista,

/ divide il primo numero inserito e il risultato intero lo divide per il numero successivo e così via,

% ritorna il resto della divisione tra il primo numero e il secondo e lo divide per l'eventuale numero successivo e così via,

++ oppure inc, incrementa un numero di una unità,

-- oppure dec, decrementa un numero di una unità.

Nella lista si possono inserire numeri, espressioni che li generano o variabili che li contengono.

Esempi:

```
(+ 1 3 5)
```

 ritorna 9,

```
(+ 1,9 2,9)
```

 ritorna 3,

```
(- 10 3 4 1)
```

 ritorna 2,

```
(* 2 3 6)
```

 ritorna 36,

```
(/ 25 2 3)
```

 ritorna 4,

```
(/ 12 (+ 3 2))
```

 ritorna 2,

```
(% 16 7)
```

 ritorna 2,

```
(% 16 7 2)
```

 ritorna 0,

```
(+ (/ 6 3) (* 2 4))
```

 ritorna 10,

```
(++ 3)
```

 ritorna 4,

```
(dec (* 3 2))
```

 ritorna 5,

se esiste una variabile `x` contenente il valore 15, `(/ x 3)` ritorna 5.

Normalmente queste operazioni producono numeri interi con un massimo di 19 cifre (per l'esattezza il numero massimo generabile con queste operazioni è 9 223 372 036 854 775 807).

Tuttavia, se almeno uno dei numeri indicati nella lista per l'operazione è superiore a questo limite, interviene automaticamente la gestione `big integer` e il risultato viene indicato come `big integer`, che è un intero con il suffisso `L` e può arrivare ad essere un intero di ben 1634 cifre.

Per poter esprimere un risultato che possa raggiungere queste dimensioni operando soltanto con interi di dimensione inferiore occorre che il primo di questi interi sia espresso come big integer, cioè con il suffisso L.

Esempi:

(`* 7000000000000 160000`) genera stack overflow e produce un risultato inaccettabile consistente in un numero negativo,
(`* 70000000000000L 160000`) ritorna `11200000000000000000L` che è un big integer di 20 cifre: si è potuto superare il limite in quanto il primo intero della lista è stato qualificato big integer,
(`+ 9223372036854775810 100`) ritorna `9223372036854775910L` che è un big integer superiore all'intero massimo generabile: ciò avviene in quanto il primo intero della lista è, sia pure di sole 3 unità, superiore all'intero massimo generabile e si apre così automaticamente la gestione big integer.

Esiste la funzione `bigint` con la quale possiamo trasformare un intero normale in big integer:

(`bigint 24`) ritorna `24L`, che è un big integer,
(`bigint a`) acquisisce come big integer il contenuto della variabile `a`,
(`bigint (int (read-line))`) acquisisce come big integer il numero che scriviamo con la tastiera⁹.

Per fare le stesse operazioni che abbiamo visto con numeri in virgola mobile, quelli con i decimali, dobbiamo usare, nell'ordine, le seguenti funzioni, che possiamo comunque usare anche per i numeri interi (senza tuttavia la possibilità di gestire i big integer):

`add` per la somma,
`sub` per la sottrazione,
`mul` per la moltiplicazione,
`div` per la divisione,
`mod` per il resto della divisione,
`inc` incrementa un numero di una unità,
`dec` decrementa un numero di una unità.

Esempi:

nel secondo esempio di prima, se scriviamo (`add 1,9 2,9`), otteniamo il risultato `4,8`,
nel quinto esempio di prima, se scriviamo (`div 25 2 3`), otteniamo il risultato `4,166666666666667`.

Queste operazioni producono numeri in virgola mobile di qualsiasi dimensione ma precisi nell'ambito di 17 cifre complessive (tra parte intera e parte decimale), virgola compresa.

Esempi:

(`div 12,5 3`) ritorna, come visto in altro modo nell'esempio di prima, `4,166666666666667`,
(`div 8967542 3`) ritorna `2989180,666666667`;

prima una cifra per la parte intera, 15 cifre per la parte decimale, per un totale di 16 più la virgola,

poi 7 cifre per la parte intera, 9 cifre per la parte decimale, per un totale di 16 più la virgola.

(`mul 9223372036854775810 1114`) ritorna `1,027483644905622e+22`, un bel numero di 23 cifre, che è tuttavia preciso per le solite 16 cifre e, dopo l'arrotondamento alla sedicesima cifra si completa con degli zeri: `102748364490562200000000`; il vero risultato, `10274836449056220252340L`, lo otterremmo sostituendo alla funzione `mul` la funzione `*`.

Altre semplici funzioni per la manipolazione di numeri sono:

`abs` che ritorna il valore assoluto di un numero,
`ceil` che arrotonda un numero in virgola mobile per eccesso,
`floor` che arrotonda un numero in virgola mobile per difetto,

⁹Tutto questo casino non lo troviamo nel normale LISP, dove, come in Python, il limite dimensionale degli interi è dato solo dall'hardware, per cui è possibile manipolare enormi interi di dimensioni illeggibili. Lutz Müller penso abbia introdotto queste novità in NewLISP per rendere più veloce il trattamento delle dimensioni sotto il limite, limite che, peraltro, corrisponde a quelli imposti da tutti gli altri linguaggi di programmazione e fogli di calcolo.

round che applica a un numero in virgola mobile il così detto arrotondamento commerciale.

6.3 Matematica

Oltre alle funzioni per l'aritmetica di base che abbiamo appena visto, NewLISP ci mette a disposizione moltissime funzioni per eseguire calcoli più complicati.

Per queste altre funzioni non vale più la distinzione tra numeri interi e numeri in virgola mobile e il risultato fornito è sempre un numero in virgola mobile (float).

Qui elencherò le funzioni di più diffusa utilità classificate in gruppi. Riferimenti completi si possono trovare nella documentazione che ho indicato all'inizio del Capitolo.

6.3.1 Matematica di base

`sqrt`

ritorna la radice quadrata di un numero,

`pow`

ritorna la potenza all'esponente indicato come secondo parametro (se non indicato calcola il quadrato)

`(pow 5 3)` ritorna 125

`(pow 5)` ritorna 25

`pow` serve anche per calcolare radici ennesime se il secondo parametro è indicato come reciproco dell'indice di radice:

`(pow 125 (div 1 3))` ritorna 5, radice cubica di 125 (attenzione ad usare, per indicare il reciproco, la funzione `div` e non `/`).

6.3.2 Trigonometria

abbiamo tutte le funzioni trigonometriche dirette (`sin` `cos` `tan`) e inverse (`acos` `asin` `atan`).

Il parametro per le dirette va espresso in radianti.

Le indirette ritornano radianti.

NewLISP non ha costanti predefinite, per cui non ha nemmeno la costante π . Possiamo tuttavia definirla noi ricorrendo a una funzione trigonometrica inversa, per esempio `acos`, in questo modo:

`(constant 'pi (acos -1))`, che immette il valore 3,141592653589793 nella costante `pi`.

In questo modo possiamo richiamare in tutto il programma il valore della costante `pi` per indicare i radianti.

`(sin (div pi 2))` ritorna 1,

`(cos (mul pi 2))` ritorna 1,

`(sin pi)` ritorna 0.

Definita la costante `pi` possiamo anche utilizzarla per trasformare i radianti in gradi con

`(div(mul <radianti> 180) pi)`

e i gradi in radianti con

`(div(mul <gradi> pi) 180).`

6.3.3 Statistica

`stats`

ritorna una lista contenente indicatori statistici relativi ad una serie di numeri; la serie di numeri, argomento della funzione, è contenuta in una lista o in un vettore;

gli indicatori statistici sono, nell'ordine:

. numero dei valori,

- . media aritmetica,
- . media aritmetica degli scarti dalla media,
- . deviazione standard campionaria,
- . varianza campionaria,
- . indice di asimmetria di Fisher,
- . indice di curtosi di Pearson.

(stats '(1 2 3 4 5 4 3 2 1)) ritorna la lista

(9 2,7777777777777778 1,135802469135802 1,394433377556793 1,9444444444444445 0,1011833743351869 -1,543431594860166)

rammento che l'indice di asimmetria di Fisher:

- . compreso tra -0,5 e 0,5 indica simmetria perfetta,
- . compreso tra -1 e -0,5 indica moderata asimmetria (coda lunga) a sinistra,
- . compreso tra 0,5 e 1 indica moderata asimmetria (coda lunga) a destra,
- . oltre -1 indica forte asimmetria a sinistra,
- . oltre 1 indica forte asimmetria a destra;

e l'indice di curtosi di Pearson:

- . uguale a 0 indica perfetta sovrapposibilità alla campana di Gauss,
- . maggiore di 0 indica una curva più appuntita (leptocurtica),
- . minore di 0 indica una curva più piatta (platicurtica).

La distribuzione dell'esempio è perfettamente simmetrica (lo si vede anche a occhio) ed è platicurtica, cioè più piatta della campana di Gauss.

`corr`

ritorna una lista contenente una serie di indicatori relativi alla correlazione esistente tra dati contenuti in due liste o vettori aventi lo stesso numero di elementi;

gli indicatori sono, nell'ordine:

- . coefficiente di correlazione r ,
- . intercetta della retta di regressione,
- . coefficiente angolare della retta di regressione,
- . test di significatività t ,
- . gradi di libertà del test di significatività t ,
- . probabilità a due code di t relativa all'ipotesi nulla.

Se abbiamo due serie di dati in altrettante variabili:

(set 'a '(7 12 22 35 46 58))

(set 'b '(8 14 20 38 50 56))

(corr a b) ritorna la lista

(0,9919130494367538 1,302724520686176 0,9899091826437941 15,63061323392482 4 9,78337158110687e-05)

dove il coefficiente di correlazione quasi pari a 1 indica perfetta correlazione

e i due dati successivi sottendono l'equazione della retta di regressione

$y = 1,302724520686176 + 0,9899091826437941x$

infine il valore prossimo a 0 della probabilità dell'ipotesi nulla conferma l'esistenza di un legame tra i dati della variabile x e quelli della variabile y .

`t-test`

ritorna una lista di valori utili per valutare la differenza tra medie campionarie; dal manuale possiamo vedere quattro possibili sintassi, qui evidenzio la più classica.

Indichiamo come argomenti della funzione i risultati di due osservazioni campionarie;

la lista ritornata dalla funzione contiene, nell'ordine:

- . media delle osservazioni del primo campione,
- . media delle osservazioni del secondo campione,
- . deviazione standard delle osservazioni del primo campione,
- . deviazione standard delle osservazioni del secondo campione,
- . valore del test t di Student,

- . gradi di libertà,
- . probabilità a due code di t relativa all'ipotesi nulla.

Se abbiamo due serie di dati

```
(set 'a '(97 86 70 68 74 71 48 65)) pulsazioni cardiache di un campione di individui presi a caso,
(set 'b '(62 72 65 60 48 52 60)) pulsazioni di individui che hanno assunto un betabloccante,
(t-test a b) ritorna la lista
(72,375 59,85714285714285 14,49076060312718 7,967194642795556 2,027114439618418 13 0,0636665985911502)
```

dal bassissimo valore della probabilità del test t corrispondente all'ipotesi nulla dobbiamo dedurre che il betabloccante fa effetto.

A proposito di test statistici abbiamo poi funzioni che, per i quattro più famosi, ritornano la probabilità (a una coda) che il valore osservato possa essere uguale o più grande nell'ipotesi nulla: prob-t è relativa al test t di Student;

alla funzione si passano come argomenti il valore del test e i gradi di libertà;

prob-f è relativa al test f di Snedecor;

gli argomenti da passare sono il valore del test e i due gradi di libertà;

prob-z è relativa al test z di Fisher;

l'unico argomento da passare è il valore del test;

prob-chi2 è relativa al test χ^2 di Pearson;

alla funzione si passano come argomenti il valore del test e i gradi di libertà.

A queste funzioni corrispondono le funzioni inverse crit-t , crit-f , crit-z e crit-chi2 , che ritornano il valore critico minimo del test corrispondente ad una certa probabilità in ipotesi nulla.

Mi sono limitato a citare le funzioni statistiche di più ricorrente utilizzo. Altre ce ne offre NewLISP e le troviamo nel manuale che ho già citato.

6.3.4 Matematica finanziaria

Delle sei funzioni che troviamo nel manuale segnalo le tre che possono essere utili a chi abbia intenzione di contrarre un prestito.

pmt

ritorna l'importo della rata necessaria per estinguere un prestito;

gli argomenti necessari, nell'ordine, sono:

- . tasso d'interesse unitario (il 5% si indica con 0,05),
- . numero delle rate,
- . importo del prestito.

```
(pmt 0,05 12 1000) ritorna -112,8254100208153
```

che è la rata (indicata in negativo, essendo un pagamento) necessaria per estinguere in 12 anni un prestito di 1000 al tasso del 5%.

pv

ritorna il valore attuale di pagamenti periodici costanti (importo del prestito);

gli argomenti necessari, nell'ordine, sono:

- . tasso d'interesse unitario (il 5% si indica con 0,05),
- . numero delle rate,
- . importo della rata.

```
(pv 0,05 12 112,82541) ritorna -999,9999998155085, cioè 1000
```

che è l'importo di un prestito che si può rimborsare in 12 anni, al tasso del 5%, con una rata di 112,82541.

nper

ritorna il numero di rate necessarie per estinguere un prestito;

gli argomenti necessari sono:

- . tasso d'interesse unitario (il 5% si indica con 0,05),
- . importo della rata,
- . importo del prestito (indicato con segno negativo).

(nper 0,05 112,82541 -1000) ritorna 12,00000000300939, cioè 12

che è il numero di rate da 112,82541 necessarie per estinguere un prestito di 1000 regolato al tasso del 5%.

Gli esempi prevedono un tasso del 5% che, salvo strozzinaggio, è un tasso su base annua, pertanto la rata si intende annuale e il relativo numero è rappresentato in anni.

Per avere il tutto rapportato a mesi, semestri, ecc., occorre inserire il tasso mensile, semestrale, ecc.

6.3.5 Array e matrici

Oltre alla funzione array per costruire array e matrici, già vista nel Capitolo 4, abbiamo una serie di funzioni per lavorare con questo tipo di dati.

append

crea un nuovo array mettendo insieme più array, senza modificare gli array di partenza;

se abbiamo l'array a (1 2 3 4) e l'array b (5 6)

(set 'c (append a b)) crea l'array c (1 2 3 4 5 6)

gli elementi da mettere insieme devono essere dello stesso tipo array;

per esempio, per aggiungere l'elemento 7 all'array c

non possiamo fare (append c 7) e nemmeno (append c '(7))

ma dobbiamo fare (append c (array 1 '(7)))

nth

ritorna un elemento di un array;

gli argomenti da indicare sono l'indice dell'elemento (partendo da 0) e l'array;

con riferimento all'array c dell'esempio di prima, (nth 1 c) ritorna 2.

mat

introduce un'operazione scalare tra due matrici o tra una matrice e un numero;

gli argomenti da indicare sono:

. la funzione aritmetica da utilizzare (+ - * /, validi, in questo caso, anche per numeri decimali),

. una matrice,

. l'altra matrice o un numero;

(mat * M1 M2) ritorna il prodotto scalare tra le matrici M1 e M2

(mat * M1 5) ritorna il prodotto scalare tra la matrice M1 e il numero 5.

multiply

esegue la moltiplicazione tra due matrici;

gli argomenti sono le due matrici da moltiplicare.

transpose

traspone la matrice indicata come argomento.

invert

calcola la matrice inversa di quella indicata come argomento.

det

calcola il determinante della matrice indicata come argomento.

6.4 Manipolazione di liste

Alle liste si applicano innanzi tutto le due funzioni `append` e `nth` che abbiamo visto per gli array: ovviamente negli argomenti, in questo caso, abbiamo liste anziché array.

Sono poi previste una serie di funzioni, applicabili sia alle liste sia alle stringhe per selezionare elementi (`select`, `find`, `first`, `last`, `rest`), sostituirli (`replace`), spostarli (`reverse`, `rotate`). Per la sintassi, peraltro molto semplice e intuitiva di queste funzioni rimando al manuale.

Se la quoted list ha come primo elemento una funzione, con `eval` possiamo valutarla.

```
'(* 3 4) ritorna '(* 3 4),  
(eval '( * 3 4)) ritorna 12.
```

6.5 Manipolazione di stringhe

Alle stringhe si applicano le funzioni `append` e `nth` che abbiamo visto per gli array e le liste: negli argomenti, in questo caso, abbiamo stringhe anziché array o liste.

Abbiamo poi tutte le funzioni per selezionare elementi (`select`, `find`, `first`, `last`, `rest`), sostituirli (`replace`), spostarli (`reverse`, `rotate`) che abbiamo visto per le liste.

Infine abbiamo funzioni riservate alle stringhe.

Oltre a quelle viste nel Capitolo 4 per le trasformazioni di tipo (`int`, `float`, `string`), ne abbiamo altre (`upper-case`, `lower-case`, rispettivamente per trasformare il contenuto della stringa in caratteri minuscoli o maiuscoli, `explode`, per trasformare la stringa in una lista di singoli caratteri) ed altre ancora, di sempre meno ricorrente utilità, che troviamo nel manuale.

Si può anche trasformare la stringa in una lista di token con la funzione `parse`.

Se a questa funzione passiamo, come solo argomento, una stringa, essa ritorna una lista di token isolandoli in presenza di uno spazio bianco, di una parentesi, di una virgola o di due punti: lo spazio bianco e i due punti si perdono nella tokenizzazione, mentre la virgola e la parentesi rimangono come token.

```
(parse "Ieri sono andato a casa di Giuseppe") ritorna ("Ieri" "sono" "andato" "a" "casa" "di" "Giuseppe")  
(parse "Ieri(mercoledì)sono andato a casa") ritorna ("Ieri" "(" "mercoledì" ")" "sono" "andato" "a" "casa")  
(parse "lunedì,martedì,mercoledì") ritorna ("lunedì" "," "martedì" "," "mercoledì")  
(parse "lunedì:martedì:mercoledì") ritorna ("lunedì" "martedì" "mercoledì")
```

Se passiamo come secondo parametro opzionale una stringa la divisione dei token avviene in presenza di questa stringa, che viene eliminata dalla stringa di partenza.

```
(parse "lunedimartedimercoledì" "edì") ritorna ("lun" "mart" "mercol" "").
```

Altra funzione che può essere utile la funzione `char` che tramuta un carattere contenuto in una stringa, se necessario identificato dal suo indice, nel corrispondente codice ASCII e viceversa.

```
(char "a") ritorna 97,  
(char 97) ritorna "a",  
(char "Vittorio" 1) ritorna 105, corrispondente a i, secondo carattere della stringa "Vittorio".
```

6.6 Manipolazione di simboli

La funzione `sym` trasforma una stringa o un numero in simbolo.

Argomento della funzione è l'elemento da trasformare, che può essere indicato anche richiamando la variabile che lo contiene.

```
(sym "pippo") ritorna pippo, che non ha più le virgolette in quanto è diventato un simbolo,  
(sym 12) ritorna 12, che non è più un numero ma un simbolo e non è utilizzabile per le funzioni aritmetiche o matematiche,  
(+ 2 (sym 3)) genera un errore per invalidità del secondo parametro.
```

6.7 Lavorare con i file

Nel paragrafo 6.1, dedicato all'Input/Output, ho mostrato le funzioni che ci consentono di lavorare con i device tastiera, per l'input, e schermo, per l'output: tastiera e schermo sono classificati da NewLISP con il numero 0 e sono i device di default.

Spesso è comodo o necessario avere altre forme di input (per esempio inserire i dati da elaborare prendendoli da un supporto che li contiene, evitando così di doverli digitare sulla tastiera) e/o altre forme di output (per fissare i risultati delle nostre elaborazioni su un supporto rileggibile e non volatile come lo schermo). In entrambi i casi dobbiamo poter lavorare con file da cui prendere o in cui inserire dati e informazioni.

Abbiamo innanzi tutto le seguenti tre funzioni che ci consentono di lavorare direttamente su file.

`write-file`

la sintassi è

`(write-file <file> <contenuto>)`

dove

`<file>` è una stringa contenente l'indirizzo e il nome del file;

`<contenuto>` è una stringa contenente ciò che vogliamo scrivere nel file.

Se il file non c'è viene creato. Se c'è viene sovrascritto, cioè il precedente contenuto va perso.

Il contenuto viene inserito senza a capo: per andare a capo dobbiamo terminare la stringa del contenuto con `\n`.

`append-file`

la sintassi è

`(append-file <file> <contenuto>)`

dove

`<file>` è una stringa contenente l'indirizzo e il nome del file;

`<contenuto>` è una stringa contenente ciò che vogliamo scrivere nel file.

Se il file non c'è viene creato. Se c'è, il contenuto viene aggiunto al contenuto precedente.

Il contenuto viene inserito senza a capo: per andare a capo dobbiamo terminare la stringa del contenuto con `\n`.

`read-file`

la sintassi è

`(read-file <file>)`

dove

`<file>` è una stringa contenente l'indirizzo e il nome del file.

Questa funzione ritorna il contenuto del file come stringa in cui compaiono anche gli eventuali simboli `\n` (eliminabili ma non sostituibili con la funzione `replace`). Essa non si presta per l'acquisizione di dati da elaborare.

Esistono modi più sofisticati per lavorare con i file e vi dobbiamo ricorrere soprattutto per l'acquisizione di dati da elaborare, in superamento dei limiti della funzione `read-file` che abbiamo appena visto. Alla base di tutto sta la funzione

`open`

questa funzione ritorna un numero intero che può essere la chiave attraverso cui interagire con un file; la sua sintassi è:

`(open <file> <modalità>)`

dove

`<file>` è una stringa contenente l'indirizzo e il nome del file;

`<modalità>` è una stringa per indicare cosa vogliamo fare con il file:

"write" o semplicemente "w" per scrivervi sopra,
"read" o semplicemente "r" per leggerlo,
"update" o semplicemente "u" per scrivervi sopra e leggerlo,
"append" o semplicemente "a" per aggiungervi dati e leggerlo.

Occorre fare molta attenzione alle modalità write e update. La modalità write, se il file indicato non c'è ancora lo crea e lo rende disponibile per accogliere informazioni, se il file indicato c'è già tutto quanto vi inseriamo va a sostituire il precedente contenuto. Quest'ultima cosa avviene anche inserendo informazioni in un file aperto in modalità update.

Se vogliamo scrivere ulteriori informazioni in un file salvando quelle che vi sono già contenute dobbiamo usare la modalità append.

Il numero intero generato dalla funzione open può essere convenientemente utilizzato per attivare un device alternativo a quello di default, come abbiamo già detto identificato con il numero 0, identificandolo più comodamente con un simbolo.

A questo scopo usiamo la funzione device che ha la seguente sintassi:

```
(device <simbolo>)
```

dove <simbolo> sottende il numero generato dalla funzione open oppure, per tornare al default (tastiera e schermo), 0.

Fintanto che è attivo il device collegato al file possiamo lavorare sul file stesso con le solite funzioni println, print e read-line che abbiamo visto per il normale Input/Output nel paragrafo 6.1.

Il device si disattiva con

```
(close <simbolo>)
```

e si torna così al default tastiera/schermo.

Un paio di esempi utili per capirci:

```
(set 'f (open "/home/vittorio/nomi" "read"))  
(device f)  
(set 'lista_nomi '())  
(while(read-line f)(set 'lista_nomi(append lista_nomi(list(sym(current-line))))))  
(close f)
```

Con questo codice ci predisponiamo a lavorare sul file nomi, che contiene l'elenco dei nomi dei miei amici, proponendoci di inserire questi nomi in una lista di simboli.

Assegniamo al simbolo f il numero corrispondente all'apertura del file per lettura e, con questo simbolo creiamo il device di lavoro.

Creiamo una variabile globale lista_nomi contenente, per intanto, una lista vuota.

Avviamo un ciclo di lettura delle righe dell'elenco contenuto nel file, trasformiamo in simbolo la stringa letta (current_line), ne facciamo una lista e andiamo ad aggiungere questa lista alla lista dei nomi nella variabile globale lista_nomi.

Chiudiamo il device.

Ora abbiamo a disposizione una variabile contenente la lista dei simboli dei nomi dei miei amici e possiamo lavorare con questi dati: rammento che in LISP tutto avviene attraverso liste e quanto abbiamo fatto si è reso necessario per trasferire in una lista il contenuto di un qualsiasi elenco che avevamo a disposizione.

```
(set 'f (open "/home/vittorio/dati.csv" "read"))  
(device f)  
(set 'lista_dati '())  
(while(read-line)(set 'lista_dati(append lista_dati(list(float(current-line))))))  
(close f)  
(stats lista_dati)
```

Con quest'altro codice facciamo un po' la stessa cosa di prima, con la differenza che l'obiettivo è quello di creare una lista contenente dati numerici da elaborare, traendoli da un file .csv.

In questo caso il codice contiene anche un esempio di come sulla nostra lista possiamo intervenire: nel caso calcolando le statistiche che caratterizzano la serie numerica ivi contenuta.

Unica precauzione: nel file .csv il separatore non può essere la virgola (rammento che il NewLISP italiano usa la virgola per separare le cifre decimali) e va eventualmente sostituito con il punto e virgola o con uno spazio, inoltre i dati devono essere elencati in colonna. Le predisposizioni possono eventualmente essere messe a punto ripassando preventivamente il file .csv con un foglio di calcolo.

7 Costruire una funzione

Può accadere che tra le funzioni preconfezionate, anche se sono tantissime, non troviamo quella che ci serve o che vogliamo fare una certa cosa in maniera diversa da come la si potrebbe fare con una funzione preconfezionata.

Per questi casi NewLISP ci mette a disposizione la funzione `define` attraverso la quale possiamo costruire noi la funzione che ci serve con la sintassi

```
(define(<nome_funzione> <parametro> <parametro>...)(<istruzione>)(<istruzione>)...)
```

dove

<nome_funzione> è il nome che diamo alla funzione per poterla richiamare,

<parametro> indica uno o più parametri richiesti perché la funzione produca risultato,

<istruzione> indica una o più istruzioni sul compito o sui compiti che la funzione debba svolgere.

Se non si indicano parametri, la funzione potrà essere richiamata senza indicare parametri, a patto che le istruzioni non abbiano bisogno di dati o li trovino disponibili altrimenti.

Esempi:

```
(define(raddoppia x)(* x 2))
```

è una funzione che serve per raddoppiare il numero indicato;

richiamandola con l'istruzione `(raddoppia 5)` essa ritorna il numero 10.

```
(define(triplica)(* y 3))
```

è una funzione che serve per triplicare un numero, ma non lo richiede;

essa, se richiamata con l'istruzione `(triplica)`, che non richiede parametri, produce un risultato solo se trova una variabile `y` contenente un numero.

```
(define(salutami)(println "Ciao, Vittorio!"))
```

è una funzione attraverso la quale mi faccio salutare dal computer;

essa, se richiamata con l'istruzione `(salutami)`, che non richiede parametri, produce il risultato, scrivendo il saluto, in quanto ha a disposizione tutto ciò che serve (la stringa da scrivere).

* * *

Esiste un elegante modo alternativo per costruire una funzione in NewLISP, che consiste nel creare una variabile che contiene una funzione scritta con un'espressione `lambda`. La sintassi è la seguente:

```
(set <simbolo> (lambda(<parametro> <parametro>...)(<istruzione>)(<istruzione>)...))
```

Le tre funzioni create prima, con questa sintassi si creerebbero così:

```
(set 'raddoppia (lambda(x) (* x 2)))
```

```
(set 'triplica (lambda() (* y 3)))
```

```
(set 'salutami (lambda() (println "Ciao, Vittorio!")))
```

* * *

Faccio un esempio meno banale.

Tra le funzioni di matematica finanziaria precostituite ne manca una di fondamentale importanza: quella per determinare il tasso equivalente, cioè il tasso per una frazione di anno equivalente ad un tasso annuo.

Quando usiamo la funzione `pmt` per determinare la rata di rimborso di un prestito, infatti, dobbiamo inserire il parametro tasso di interesse riferito al periodo di rata: nell'esempio che avevamo visto l'ipotesi era di un prestito di 12 anni con rata di rimborso annuale e avevamo indicato il tasso annuo 0,05 (5%).

Ma supponiamo ora che il prestito duri un solo anno e sia da rimborsare in dodici rate mensili. In questo caso dobbiamo inserire il tasso mensile, anche se, quando concordiamo le condizioni del prestito, parliamo di tasso annuo. In genere, concordato, per esempio, il tasso annuo del 5%, per semplicità si determina il tasso mensile dividendo per 12 il tasso annuo. Si tratta di una pratica semplificatoria tollerabile se siamo in presenza di tassi bassi e di prestiti di importi modesti, ma si compie un grossolano errore, tra l'altro andando contro la legge che regola il TAEG: in regime di capitalizzazione composta mensile, infatti, non è vero che il tasso mensile corrisponde a $1/12$ del tasso annuo e, in presenza di tassi e importi di prestito non modesti, la differenza si nota.

Per fare le cose per bene, pertanto, conviene che disponiamo di una funzione che calcola i tassi equivalenti utilizzando la nota formula $i_k = (1 + i)^{\frac{1}{k}} - 1$ che fornisce il tasso per $\frac{1}{k}$ di anno equivalente al tasso annuo i .

Ce la possiamo costruire con il seguente codice

```
(define (tasso_equivalente i k) (sub (pow (add 1 i) (div 1 k)) 1))
```

* * *

Le funzioni che ci servono possiamo costruirle nel corso del programma e richiamarle quando le utilizziamo.

Ma, se la funzione è laboriosa da costruire, come comincia ad esserlo quella appena vista del tasso equivalente, ci conviene salvarla, con un nome sufficientemente descrittivo, in un file con estensione `.lsp` e in una directory dedicata.

Nel mio sistema Linux, per esempio, creo nella mia home una directory chiamata `funzioni_newlisp`. Scrivo il codice per creare la funzione `tasso_equivalente` con un editor di testo e salvo il file

```
tasso_equivalente.lsp
```

nella directory `funzioni_newlisp`.

Ho così a disposizione la funzione per richiamarla quando e dove serve semplicemente caricandola nella shell interattiva o nel programma, senza più riscriverla, con l'istruzione

```
(load "/home/vittorio/funzioni_newlisp/tasso_equivalente.lsp").
```

Data questa istruzione possiamo calcolare la rata mensile del nostro prestito di 1000 con l'istruzione

```
(pmt (tasso_equivalente 0,05 12) 12 1000) che ritorna -85,55659945661426
```

solo lievemente inferiore al valore che otterremmo con la formula semplificata per ottenere il tasso mensile dividendo per 12 il tasso annuo

```
(pmt (div 0,05 12) 12 1000) ritorna -85,60748178846745.
```

Se però abbiamo a che fare con un mutuo per la casa di 250000, al tasso del 6%, rimborsabile in 15 anni con rate semestrali

```
(pmt (tasso_equivalente 0,06 2) 30 250000) ritorna -12682,8735102168
```

```
(pmt (div 0,06 2) 30 250000) ritorna -12754,81483006314
```

con una differenza di 72 per ogni rata, che, proiettata su 30 rate genera un maggiore rimborso di 2160 che non sarebbe dovuto.

8 Interattività con l'utente

Spesso, soprattutto quando si tratta di programmi destinati ad eseguire calcoli, diventa utile creare un programma che astragga dai parametri necessari alle varie funzioni per ottenere risultati e che chieda questi parametri all'utente in sede di esecuzione.

Il seguente programma, che scriviamo con un editor di testo e salviamo in un file `cerchio.lsp`, calcola circonferenza e area del cerchio in presenza del raggio 2,5.

```
(constant 'pi (acos -1))  
(mul 2,5 pi 2)  
(mul (pow 2,5 2) pi)
```

ma ha due difetti: innanzi tutto tiene per sé i risultati, inoltre è fissamente dedicato ad un cerchio di raggio 2,5 e dovremmo riscriverlo ogni volta che vogliamo fare i calcoli con un raggio diverso.

Al primo difetto possiamo ovviare utilizzando la funzione per l'output `println`. Se riscriviamo il programma in questo modo

```
(constant 'pi (acos -1))  
(println (mul 2,5 pi 2))  
(println (mul (pow 2,5 2) pi))  
(exit)
```

otteniamo la stampa del valore della circonferenza e del valore dell'area del cerchio e l'uscita dall'interprete grazie alla funzione `exit`.

Se ci posizioniamo nella directory dove è memorizzato il file `cerchio.lsp` e scriviamo il comando `newlisp cerchio.lsp`

otteniamo in risposta

```
15,70796326794897
```

```
19,63495408493621
```

nell'ordine circonferenza ed area del cerchio di raggio 2,5.

Al secondo difetto ovviamo ricorrendo alla funzione `println` per l'output di un messaggio di richiesta all'utente e successivamente alla funzione `read-line` per l'input di quanto l'utente scriverà sulla tastiera. Il nostro programma diventa

```
(constant 'pi (acos -1))  
(println "Inserisci il raggio del cerchio")  
(set 'raggio (float(read-line)))  
(println (mul raggio pi 2))  
(println (mul (pow raggio 2) pi))  
(exit)
```

Se lanciamo questo programma ci viene chiesto di indicare il raggio del cerchio e, una volta che l'avremo digitato sulla tastiera e premuto INVIO, otterremo circonferenza e area del cerchio avente il raggio da noi indicato.

Se vogliamo perfezionare il tutto, accontentandoci dei risultati espressi con sole due cifre decimali, possiamo modificare così il nostro programmino `cerchio.lsp`:

```
(constant 'pi (acos -1))  
(println "Inserisci il raggio del cerchio")  
(set 'raggio (float(read-line)))  
(set 'circonferenza (mul raggio pi 2))  
(set 'area (mul (pow raggio 2) pi))  
(println "Per un cerchio di raggio " raggio)  
(println "la circonferenza è:" (format {%15.2f} circonferenza))  
(println "e l'area è:          " (format {%15.2f} area))  
(exit)
```

9 Strutture di controllo

NewLISP ci offre il modo di controllare l'esecuzione del programma attraverso funzioni per condizionare l'esecuzione di una o più istruzioni al verificarsi di certe condizioni oppure per la ripetizione dell'esecuzione di una o più istruzioni.

Per esprimere condizioni abbiamo a disposizione le funzioni di confronto

< minore
> maggiore
= uguale
<= minore o uguale
>= maggiore o uguale
!= diverso

9.1 Esecuzione condizionale

when

la sua sintassi è

```
(when <condizione> <istruzione>)
```

se la condizione è vera viene eseguita l'istruzione, altrimenti non succede nulla e il programma continua con le altre istruzioni, se ce ne sono, non dipendenti dalla condizione.

```
(when (> x y) (println "SI"))
```

```
(println "Ciao")
```

se il valore della variabile *x* è superiore a quello della variabile *y* in output abbiamo la scritta *SI* e la scritta *Ciao*, in caso contrario in output abbiamo la sola scritta *Ciao*.

if

la cui sintassi prevede tre possibilità

```
(if <condizione> <istruzione>)
```

```
(if <condizione> <istruzione> <istruzione>)
```

```
(if <condizione> <istruzione> <condizione> <istruzione> ...)
```

Nel primo caso otteniamo, tali e quali, gli effetti della precedente funzione *when*.

Nel secondo caso, se la condizione è vera viene eseguita la prima istruzione, altrimenti viene eseguita la seconda (è ciò che in altri linguaggi si ottiene con l'istruzione *if ... else*).

```
(if (> x y) (println "SI") (println "NO"))
```

```
(println "Ciao")
```

se il valore della variabile *x* è superiore a quello della variabile *y* in output abbiamo la scritta *SI* e la scritta *Ciao*, in caso contrario in output abbiamo la scritta *NO* e la scritta *Ciao*.

Nel terzo caso possiamo abbinare quante istruzioni vogliamo ad altrettante condizioni.

```
(if
```

```
(> x y) (println "maggiore")
```

```
(< x y) (println "minore")
```

```
(= x y) (println "uguale"))
```

```
(println "Ciao")
```

se il valore della variabile *x* è superiore a quello della variabile *y* in output abbiamo la scritta *maggiore* e la scritta *Ciao*, se il valore della variabile *x* è inferiore a quello della variabile *y* in output abbiamo la scritta *minore* e la scritta *Ciao*, se il valore della variabile *x* è uguale a quello della variabile *y* in output abbiamo la scritta *uguale* e la scritta *Ciao*.

9.2 Ripetizione

NewLISP prevede molte funzioni per il looping. Si va da quelle che ripetono azioni per ciascun elemento di una lista o di una stringa a quelle che ripetono azioni per un numero prefissato di volte, da quelle che ripetono fino a quando succede qualche cosa a quelle che ripetono fino a quando si verificano certe condizioni.

for

è la classica per ottenere una ripetizione per un numero prefissato di volte; la sintassi è

```
(for(<simbolo> <da> <a>)<istruzione> <istruzione>...)
```

dove

<simbolo> crea una variabile locale che funge da contatore e, volendo, da variabile

<da> è il numero intero da cui si comincia a contare

<a> è il numero intero con cui si finisce di contare

<istruzione> è ciò che si deve fare e ce ne possono essere più di una.

(for (i 1 5) (println "Ciao")) scrive cinque volte la parola Ciao.

(for (i 1 5) (println (pow i))) scrive i numeri 1 4 9 16 25.

dotimes

altra per ottenere una ripetizione per un numero prefissato di volte; la sintassi è

```
(dotimes (<simbolo> <numero>) <istruzione> <istruzione> ... )
```

dove

<simbolo> crea una variabile locale che funge da contatore (partendo da 0)

<numero> è il numero intero di volte per cui si devono eseguire le istruzioni

<istruzione> è ciò che si deve fare e ce ne possono essere più di una.

(dotimes (i 5) (println "Ciao")) scrive cinque volte la parola Ciao.

(dotimes (i 5) (println (pow (+ i 1)))) scrive i numeri 1 4 9 16 25.

dolist

applica una o più istruzioni ad ogni elemento di una lista o array; la sintassi è:

```
(dolist(<simbolo> <lista>) <istruzione> <istruzione> ...)
```

dove

<simbolo> crea una variabile locale che assume via via i valori degli elementi della lista

<lista> è la lista dei valori da scorrere, può essere richiamata se contenuta in una variabile

<istruzione> è ciò che si deve fare e ce ne possono essere più di una.

(set 'l '())(dolist (i '(1 2 3)) (set 'l (append l (list (pow i 3))))) inserisce in una lista l i valori 1 8 27.

dostring

applica una o più istruzioni ad ogni elemento di una stringa; la sintassi è:

```
(dostring (<simbolo> <stringa>) <istruzione> <istruzione> ...)
```

dove

<simbolo> crea una variabile locale che assume i codici ASCII degli elementi della stringa

<stringa> è la stringa da scorrere, può essere richiamata se contenuta in una variabile

<istruzione> è ciò che si deve fare e ce ne possono essere più di una.

(dostring (i "abc") (println i)) scrive i numeri 97 98 99.

until

esegue una o più istruzioni fino a quando diventa vera una condizione; la sintassi è:

```
(until <condizione> <istruzione> <istruzione> ...)
```

```
(set 'contatore 0)
```

```
(until (= contatore 4) (println "Ciao") (inc contatore)) scrive 4 volte Ciao
```

`while`

esegue una o più istruzioni fino a quando è vera una condizione; la sintassi è:

```
(while <condizione> <istruzione> <istruzione> ...)
```

```
(set 'contatore 0)
```

```
(while (< contatore 4) (println "Ciao") (inc contatore))
```

 scrive 4 volte Ciao